

Phase 2 Completion Report

Booking & Checkout

Project: Bliss Rent — Dubai Airport Rental Platform (bliss.rent)

Phase: 2 of N — Booking & Checkout (built on the approved Phase 1 vehicle search)

Prepared for: Development lead

Date: August 25, 2026

Repository: dubai-airport-rental (local + Desktop copy)

Commit: `Phase 2: booking & checkout`

✓ Build clean · ✓ Lint clean · ✓ 78/78 tests passing · ✓ Migration validated locally
Scope followed exactly as specified — no admin dashboard, WhatsApp, complaints, or deployment work included.

1 What Was Built

The full booking & checkout journey: **Vehicle Details** → **Continue Booking** → **Customer Details** → **Driver Details** → **Booking Summary** → **Payment** → **Booking Confirmation**, exactly as scoped. A customer can now create a real rental booking end-to-end through the website. Every step is a real route (not just component state), so browser refresh and the back button behave correctly.

Key architectural decision — guest checkout, no login wall: Phase 0 deliberately left `customers.auth_user_id` nullable and gave `customers.email` a case-insensitive unique index "to support matching a returning guest by email later." Phase 2 uses exactly that: a customer is matched or created by email, with no Supabase Auth session required to book. This is documented in `docs/ARCHITECTURE.md` and is why booking creation runs through privileged, server-side functions rather than direct client writes.

2 Customer Booking Flow

Step	Route	What happens
1. Continue Booking	<code>/vehicles/:id</code>	Button now navigates into checkout, carrying dates/locations in the URL
2. Customer Details	<code>/checkout/:id/customer</code>	Full name, email, phone — validated client-side for UX
3. Driver Details	<code>/checkout/:id/driver</code>	Driver's own info (customer supplies the driver — never a company driver)
4. Booking Summary	<code>/checkout/:id/summary</code>	Everything reviewed before payment; "Confirm" actually creates the booking
5. Payment	<code>/checkout/:id/payment/:bookingId</code>	TEST ONLY simulated payment, clearly labeled
6. Confirmation	<code>/checkout/:id/confirmation/:bookingId</code>	Reference, full details, explicit "not yet handed over" note

Form data (customer/driver) is persisted to `sessionStorage` per vehicle (`useCheckoutDraft.ts`) so it survives a refresh or back-navigation without needing a server session.

3 Customer Fields

Exactly what the brief asked for, nothing invented: **full name, email, phone** (phone optional, matching the nullable DB column). No account/password is required.

4 Driver Fields

Matches the existing `drivers` table exactly: full name, date of birth, license number, license country, license expiry. Two business-rule validations were added at the server layer (authoritative, in the Edge Function): the driver must be **at least 18** at the start of the rental, and the license must **still be valid through the end of the rental** — both checked against the real rental dates, not just "today".

Driver documents — deliberately deferred. The Phase 0 private `driver-documents` storage bucket still exists and is untouched, but Phase 2 does not build a document-upload step. The brief said "where required", not "required" — license/ID are checked physically at handover instead. Building a guest-safe private upload would need a third privileged Edge Function for a nice-to-have that isn't necessary to create a booking, so it was consciously left out rather than half-built. Flagging this explicitly rather than leaving it silent.

5 Booking Logic

- Dates, vehicle, and pickup/drop-off locations are validated.
- Customer and driver fields are validated server-side (authoritative — not just the client-side copy).
- The vehicle's real-time availability is re-checked at the database layer, atomically, at the moment of insert — see Double-Booking Protection below.
- The price is recalculated server-side (never trusted from the browser — see Pricing Logic).
- Customer, booking, driver, and payment (status `pending`) are created together in **one atomic transaction** — if anything fails, nothing is left half-created.
- A short booking reference (e.g. `BLS-4E21E88E`) is generated and returned.

Double-booking protection

The Phase 0 `bookings_no_overlap` exclusion constraint is what actually makes this race-safe — not a "check then insert" (which can still race), but a single atomic constraint Postgres enforces on the insert itself. If another booking for the same vehicle/date-range commits in between search and checkout, the insert raises `exclusion_violation` (SQLSTATE 23P01); the Edge Function catches this and returns a clean **"someone else just booked this vehicle"** message with a link back to search — never a fake success, and validated locally (see Tests).

6 Pricing Logic

One authoritative pricing path, as required. The price is computed by calling the exact same `resolveTermForDays / quoteForDays` functions from `src/lib/pricing.ts` that Phase 1 already built and tested for on-site display — not a second implementation. The difference is *where* it runs: for display it runs in the browser (an estimate); for an actual booking it is called again inside the `create-booking` Edge Function, server-side, using freshly-fetched pricing rows. The browser never sends a price at all — `CreateBookingRequest` has no price field, structurally.

7 Payment Architecture

A clean, swappable contract: `create-booking` creates a `payments` row (status `pending`); a separate `confirm-payment` Edge Function resolves it to `paid` or `failed` and, on success, advances the booking to `confirmed`.

TEST ONLY — no real gateway integrated. The business hasn't selected a payment provider yet, so `supabase/functions/_shared/testPaymentProvider.ts` simulates one using a well-known test-card convention (a card ending in `0002` simulates a decline; anything else simulates success) — labeled TEST ONLY in the code and in the checkout UI itself, never presented as a real charge. The outcome is decided **server-side**, never by trusting a "succeeded" flag from the browser. Swapping in a real gateway later means replacing this one file's decision logic — nothing else in the flow changes.

`confirm_payment` is **idempotent**: a duplicate submit (double-click, resubmitted form) for an already-resolved payment returns the same result rather than erroring or double-processing.

8 Database Changes

No new tables — Phase 0/1 schema is reused entirely, as required. Two new SECURITY DEFINER functions only:

Function	Purpose
<code>create_booking(...)</code>	Atomically creates/updates the guest customer, then the booking, driver, and pending payment
<code>confirm_payment(...)</code>	Atomically resolves a payment and advances the booking on success; idempotent

Both are explicitly `revoke` d from `public / anon / authenticated` and granted only to `service_role` — closing Postgres's default "grant execute to PUBLIC" behavior, which would otherwise let any browser client call them directly with a fabricated price.

9 Migrations

One new migration: `supabase/migrations/20260826000000_phase2_booking_checkout.sql`.

```
create or replace function create_booking(
  p_vehicle_id uuid, p_pickup_location_id uuid, p_dropoff_location_id uuid,
  p_start_date date, p_end_date date, p_term pricing_term,
  p_unit_price numeric, p_total_price numeric, p_currency text,
  p_customer_full_name text, p_customer_email text, p_customer_phone text,
  p_driver_full_name text, p_driver_date_of_birth date,
  p_driver_license number text, p_driver_license_country text,
  p_driver_license_expiry date, p_payment_provider text
) returns table (...) language plpgsql security definer
set search_path = public as $$
begin
  -- validates dates/price/customer fields, locks + checks the vehicle,
  -- validates locations, upserts the guest customer by email,
  -- inserts booking (protected by bookings_no_overlap), driver, payment
end;
$$;

revoke execute on function create_booking(...) from public, anon, authenticated;
grant execute on function create_booking(...) to service_role;
-- same revoke/grant pattern for confirm_payment(...)
```

Action needed: this migration has not been applied to the live Supabase project — network access remains blocked from this environment (same limitation as Phase 0/1). Open `supabase/migrations/20260826000000_phase2_booking_checkout.sql`, select all, copy, and run it in the Supabase Dashboard SQL Editor.

10 Security Changes

- Both new SQL functions are locked to `service_role` only (see Database Changes) — the anon key can never call them directly.
- Two new Edge Functions (`create-booking` , `confirm-payment`) hold the service-role key server-side only; it is never present in any frontend code or bundle.
- The browser never sends a price or an availability claim — both are recalculated authoritatively server-side.

- No driver documents, customer private data, payment secrets, or the service-role key are exposed in any frontend response.
- The confirmation page cannot be used to view someone else's booking — it only reads from this browser's own `sessionStorage`, captured at the moment of that booking's own payment confirmation; there is no ID-guessable lookup endpoint.

11 Tests

Layer	Coverage	Result
Server validation (<code>_shared/validation.ts</code>)	Customer & driver field rules, including age/license-expiry-vs-rental-dates	PASS
TEST payment provider	Approve/decline decision logic	PASS
<code>create-booking/logic.ts</code>	Successful booking (asserts server-computed price used, not any caller-supplied value), invalid dates, invalid customer/driver, no pricing, double-booking race mapping, vehicle-not-found, vehicle-unavailable	PASS
<code>confirm-payment/logic.ts</code>	Successful payment, simulated decline, missing payment id, payment-not-found, duplicate-submit idempotency	PASS
Client-side validation (UX only)	Same rule set, mirrored for immediate feedback	PASS
<code>checkoutStorage.ts</code>	Round-trip persistence, missing/corrupted data handled gracefully	PASS
<code>ConfirmationPage</code>	Shows correct details for a real confirmation; shows an honest not-found state (never fake data) for an unknown booking; never claims physical handover	PASS

78 of 78 automated tests passing (49 new this phase).

Manual database validation (local Postgres 16, both migrations + seed applied)

Scenario	Result
Successful booking creation — correct row counts, correct reference format	CORRECT
Double-booking the same vehicle/overlapping dates → <code>exclusion_violation</code> , and the whole attempt (including the customer insert) rolls back atomically	CORRECT
Booking a <code>maintenance</code> -status vehicle → rejected	CORRECT
Booking a non-existent vehicle → rejected	CORRECT
Returning guest (same email) reuses the same customer row	CORRECT
<code>confirm_payment</code> paid path → booking becomes <code>confirmed</code>	CORRECT
<code>confirm_payment</code> called twice (duplicate submit) → idempotent, no duplicate side effects	CORRECT
<code>confirm_payment</code> failed path → booking stays <code>pending_payment</code>	CORRECT
Cancelling a booking frees the vehicle for the same dates again	CORRECT
<code>anon</code> role cannot call <code>create_booking</code> or <code>confirm_payment</code> at all	CORRECT

Known test gap, flagged rather than hidden: the actual Deno HTTP handlers (`index.ts` in each Edge Function — thin wiring only, ~30 lines each) could not be executed in this sandbox (no Deno runtime available, and no network access to the live Supabase project). All of the real logic those files call (`logic.ts`) is fully unit-tested with a mocked Supabase client, and the SQL layer is fully tested against local Postgres. What's unverified by automation is only the thin HTTP/CORS/env-var wiring — see Manual Testing below for how to verify that once deployed.

12 Build Result

CLEAN `npm run build` completes with no errors. 103 modules transformed.

13 Lint Result

CLEAN `npm run lint` (oxlint) reports zero issues.

14 Manual Testing Performed

Full flow logic exercised via the automated test suite and local database validation above (search → vehicle → continue booking → customer → driver → summary → payment → confirmation, invalid dates, unavailable vehicle, overlapping booking, payment failure, duplicate submit). Full interactive click-through in a real browser against the live Supabase project could not be done from this sandboxed

environment — see section 17 for exactly what to click through once the migration is applied.

15 Files Changed

36 files changed, 2,671 insertions, 14 deletions.

Category	Files
New checkout pages/logic (13)	CustomerDetailsPage, DriverDetailsPage, BookingSummaryPage, PaymentPage, ConfirmationPage, CheckoutStepLayout, CheckoutSummaryCard, CheckoutLoadGate, useCheckoutContext, useCheckoutDraft, checkoutApi, checkoutStorage, validation (+3 test files)
New Edge Functions (8 files)	create-booking (index+logic+test), confirm-payment (index+logic+test), _shared (cors, errors, validation+test, testPaymentProvider+test)
New migration	supabase/migrations/20260826000000_phase2_booking_checkout.sql
Modified	App.tsx (routes), types/domain.ts (Customer/Driver/Checkout/Confirmation types), VehicleDetailPage.tsx (wired Continue Booking), NavBar/Footer/index.html (branding → Bliss Rent)

16 Unresolved Issues

- Driver document upload not built — deliberate scope decision, see section 4.
- An abandoned `pending_payment` booking (customer leaves without paying) currently holds the vehicle's date range indefinitely — there is no expiry/cleanup job. Not part of the Phase 2 brief's explicit scope; flagged as a good candidate for a later phase (e.g. a scheduled job cancelling stale pending bookings).
- Edge Function HTTP wiring (`index.ts`) is untested by automation in this sandbox — see the callout in section 11.

17 Exact Manual Steps You Need To Perform

1. **Run the new migration.** Open `supabase/migrations/20260826000000_phase2_booking_checkout.sql` , select all, copy, and run it in the Supabase Dashboard SQL Editor.
2. **Deploy the two Edge Functions** using the Supabase CLI from your machine: `supabase functions deploy create-booking` and `supabase functions deploy confirm-payment` (requires the project to be linked — see `supabase/config.toml`).
3. **Set the service-role secret** for the functions (never as a `VITE_` variable): `supabase secrets set SUPABASE_SERVICE_ROLE_KEY=... SUPABASE_URL=...` .
4. **Add at least one vehicle with pricing** in the database (via SQL Editor or a later admin dashboard) — Phase 0/1 seed data intentionally has no vehicles yet, so checkout can't be tested end-to-end until one exists.
5. Click through manually: search → vehicle → Continue Booking → customer details → driver details → summary → payment (try both a normal test card and one ending in `0002` to see the decline path) → confirmation.
6. Also try: submitting with invalid dates, an unavailable vehicle, two overlapping bookings for the same vehicle in two tabs, refreshing mid-checkout, using the back button, and double-clicking "Confirm & Pay".

✔ **Phase 2 complete and verified end-to-end (server-side logic + local database). Stopping here per scope — no admin dashboard, fleet management, WhatsApp integration, complaint management, advanced reporting, or production deployment has been started. Awaiting confirmation before Phase 3.**